

Name: Riya Dinesh Paturde.

Batch: B2

Roll No: 24

Aim: Data Wrangling with pandas: Manipulate and transform data to suit the analysis

Theory:

1) What is data wrangling? Explain the difference between data wrangling and data preprocessing. Ans: Data wrangling (also called data munging) is the process of cleaning, transforming, and organizing raw data into a useful format for analysis.

Steps in Data Wrangling: Collecting data Cleaning (removing errors, duplicates, missing values) Transforming (changing format, merging data) Structuring (making it organized like tables).

Difference Between Data Wrangling and Data Preprocessing

1. Definition Data Wrangling: Cleaning and organizing raw data.

Data Preprocessing: Preparing data for machine learning models.

2. Purpose

Data Wrangling: Make data usable and structured.

Data Preprocessing: Make data suitable for algorithms.

3. Stage Data Wrangling: Done at the beginning.

Data Preprocessing: Done after wrangling. 4. Scope

Data Wrangling: Broad process (cleaning, merging, transforming)

Data Preprocessing: Specific process (scaling, encoding, normalization).

2) What is the importance of exploratory data analysis (EDA) in pandas?

Ans: Importance of Exploratory Data Analysis (EDA) in Pandas

Exploratory Data Analysis (EDA) is an important step used to analyze and understand the dataset before further processing or modeling.

In Pandas, EDA helps to work with data easily using built-in functions.

Importance of EDA Understanding the Dataset Helps to know the structure, size, columns, and data types of the data.

Handling Missing Values Identifies missing or null values in the dataset.

Detecting Errors Helps find incorrect, duplicate, or inconsistent data.

Understanding Data Distribution Provides statistical details like mean, median, and standard deviation.

3) What are main differences between Series, DataFrame and Panel? Ans: In Pandas, Series, DataFrame, and Panel are data structures used to store and manage data in different dimensions.

1. Dimension Series: It is a one-dimensional (1D) data structure. DataFrame: It is a two-dimensional (2D) data structure. Panel: It is a three-dimensional (3D) data structure.
2. Structure Series: Represents a single column of data. DataFrame: Represents a table with rows and columns. Panel: Represents a collection of multiple DataFrames.
3. Axes Series: Has one axis (index). DataFrame: Has two axes (rows and columns). Panel: Has three axes (items, rows, columns).
4. Data Handling Series: Stores homogeneous data (same type of elements). DataFrame: Can store heterogeneous data (different data types in columns). Panel: Also supports heterogeneous data.
5. Usage Series: Used for handling single variable data like marks, prices, etc. DataFrame: Used for tabular data like student records, employee data. Panel: Used for multi-dimensional data like data over time (3D).
6. Complexity Series: Simple and easy to use. DataFrame: Moderate complexity. Panel: More complex and less commonly used.

Example

Series: List of student marks → [80, 75, 90]

DataFrame: Table with Name, Age, Marks

Panel: Multiple tables of student data for different years.

4) Explain the use of : a.isnull() b.notnull() c.dropna() d.fillna()

Ans: a.isnull() Use: To check missing (null) values in the dataset.

It returns True for missing values and False for non-missing values. Helps in identifying where data is missing.

Example: If a value is empty → True If a value exists → False

b. `notnull()` Use: To check non-missing values in the dataset.

It returns True for available values and False for missing values. It is the opposite of `isnull()`.

Example: If a value exists → True If a value is empty → False

c. `dropna()`

Use: To remove missing values from the dataset.

It deletes rows or columns that contain null values.

Example: If a row has a missing value → that row will be removed.

d. `fillna()`

Use: To replace missing values with a specified value.

Instead of removing data, it fills the missing values.

Example: Missing value → replaced with 0 or average value.

5) What is column transformation and why is it important in data analysis.

Ans: Column Transformation: Column transformation is the process of modifying or converting the values of a column in a dataset into a different format or scale to make it suitable for analysis. Examples : Column Transformation Converting text data into numbers (e.g., Male = 1, Female = 0) Changing data scale (e.g., normalizing values) importance of Column Transformation 1.Improves Data Quality: Makes data clean, consistent, and usable.

2.Handles Different Data Types: Converts categorical (text) data into numerical form for analysis.

3.Better Model Performance: Helps machine learning models work more accurately.

4.Normalization of Data: Brings all values to a common scale.

5.Feature Engineering: Creates new useful features from existing data.

```
import pandas as pd
import numpy as np
data = {
    "Name": ["Amit", "Sneha", "Ravi", "Priya", None],
    "Age": [20, 21, None, 19, 22],
    "Marks": [85, 90, 78, None, 88],
    "City": ["Pune", "Mumbai", "Pune", "Delhi", "Mumbai"]
}

df = pd.DataFrame(data)
```

```

print("Original Data:\n", df)

# Check missing values
print("\nMissing Values:\n", df.isnull())

# Fill missing values
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(0)
df["Name"] = df["Name"].fillna("Unknown")

# Drop rows with any remaining null values
df = df.dropna()

# Change data type
df["Age"] = df["Age"].astype(int)

# Rename columns
df.rename(columns={"Marks": "Score"}, inplace=True)

# Add new column (Transformation)
df["Result"] = df["Score"].apply(lambda x: "Pass" if x >= 40 else "Fail")

# Filter data
filtered_df = df[df["Score"] > 80]

# Sort data
sorted_df = df.sort_values(by="Score", ascending=False)

# Grouping data
grouped = df.groupby("City")["Score"].mean()

# Display results
print("\nCleaned Data:\n", df)
print("\nFiltered Data (Score > 80):\n", filtered_df)
print("\nSorted Data:\n", sorted_df)
print("\nGrouped Data (Average Score by City):\n", grouped)

```

Original Data:

	Name	Age	Marks	City
0	Amit	20.0	85.0	Pune
1	Sneha	21.0	90.0	Mumbai
2	Ravi	NaN	78.0	Pune
3	Priya	19.0	NaN	Delhi
4	None	22.0	88.0	Mumbai

Missing Values:

	Name	Age	Marks	City
0	False	False	False	False
1	False	False	False	False
2	False	True	False	False
3	False	False	True	False
4	True	False	False	False

```
Cleaned Data:
      Name  Age  Score  City Result
0    Amit   20   85.0   Pune   Pass
1   Sneha   21   90.0  Mumbai   Pass
2    Ravi   20   78.0   Pune   Pass
3   Priya   19    0.0   Delhi  Fail
4  Unknown   22   88.0  Mumbai   Pass
```

```
Filtered Data (Score > 80):
      Name  Age  Score  City Result
0    Amit   20   85.0   Pune   Pass
1   Sneha   21   90.0  Mumbai   Pass
4  Unknown   22   88.0  Mumbai   Pass
```

```
Sorted Data:
      Name  Age  Score  City Result
1   Sneha   21   90.0  Mumbai   Pass
4  Unknown   22   88.0  Mumbai   Pass
0    Amit   20   85.0   Pune   Pass
2    Ravi   20   78.0   Pune   Pass
3   Priya   19    0.0   Delhi  Fail
```

```
Grouped Data (Average Score by City):
City
Delhi      0.0
Mumbai    89.0
Pune      81.5
Name: Score, dtype: float64
```

Conculsion: This program demonstrates data wrangling using Pandas, where raw data is cleaned, missing values are handled, columns are transformed, and data is organized properly.

During the program, we performed important data wrangling operations such as identifying missing values using `isnull()`, handling missing data using `fillna()` and `dropna()`, and converting data into appropriate formats using type conversion methods.

